Week 2, Lecture 1 Handout: CSS Syntax & The Box Model Foundations

---

🎨 What is CSS? (Cascading Style Sheets)

While HTML builds the raw structure and skeleton of a webpage, **CSS** serves as the presentation and styling layer. It dictates colors, fonts, spacing, and layouts. The word **Cascading** is critical: it means styles apply from the top down, and specific rules can override general rules based on priority and inheritance.

The Three Ways to Write CSS

1. **Inline CSS**: Written directly inside an HTML tag using the `style` attribute. *(Avoided in production because it makes code messy and hard to maintain).* [6, 7, 8]
2. **Internal CSS**: Written inside a `<style>` block within the HTML `<head>`. *(Used occasionally for single-page applications).*
3. **External CSS (Industry Standard)**: Written in a completely separate `.css` file and linked inside the HTML `<head>` using the `<link>` tag. This separates structure from presentation entirely.

🛠️ The Anatomy of a CSS Rule

To style an element, you write a **CSS Rule Set**. It consists of a selector and a declaration block:

```
/* Selector targets the HTML element */
h1 {
    /* Property: Value; forms a Declaration */
    color: #2c3e50;
    font-size: 32px;
    text-align: center;
}
```

Essential Basic Selectors

- **Element Selector**: Targets every instance of an HTML tag on the page (e.g., `p { color: red; }`).
- **Class Selector**: Targets specific elements containing a matching `class=""` attribute. Class names are reusable across multiple elements and always start with a dot `.` in CSS (e.g., `.profile-card { background: white; }`).
- **ID Selector**: Targets a single, unique element containing a matching `id=""` attribute. An ID can only be used once per page and always starts with a hashtag `#` in CSS (e.g., `#main-submit-btn { width: 100%; }`).

📦 The Core Concept: The CSS Box Model

According to the browser engine, **every single element on a webpage is a rectangular box.** Even if an element looks circular on screen, its layout footprint is still a square block. [23]

The Box Model is the structural calculation engine that defines the size and spacing of these elements. It consists of four distinct layers from the inside out: [24, 25, 26]

1. **Content**: The central area where text, images, or child elements reside. Managed by `width` and `height` properties. [27]
2. **Padding**: The transparent clearing space **inside** the element box, pushing the content away from its border. Managed by properties like `padding`, `padding-top`, or `padding-left`. [28, 29]
3. **Border**: The outer protective line wrapping around the padding and content. Controlled via `border-width`, `border-style` (like `solid` or `dashed`), and `border-color`. [30]
4. **Margin**: The transparent spacing **outside** the element box, used to push neighboring elements away from it. Managed by properties like `margin`, `margin-bottom`, or `margin-right`. [31, 32]

⚠️ The Secret Weapon: `box-sizing: border-box;`

By default, browsers calculate the total width of an element like this:

$$\text{Total Width} = \text{Width} + \text{Left Padding} + \text{Right Padding} + \text{Left Border} + \text{Right Border}$$

If you set an element's width to `300px` and add `20px` of padding, the browser renders it at `340px`, which breaks responsive layouts.

The Professional Fix: Senior developers reset this behavior on day one using the universal selector (`*`) to ensure width calculations include padding and borders seamlessly:

```
* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}
```

🏠 Homework Assignment: Styling the Capstone Profile

Objective: Link an external stylesheet to your HTML Capstone Project file and apply professional typography, color harmony, and proper Box Model spacing constraints.

Task Instructions

1. Open your Week 1 `project.html` file inside **VS Code**.
2. Inside your HTML `<head>`, link a new external file named `style.css` using the `<link>` tag:

```
<link rel="stylesheet" href="style.css"
```

3. Create the `style.css` file in the same folder and add the professional layout reset (`* { box-sizing: border-box; margin: 0; padding: 0; }`) at the very top.
4. Target the page `<body>` and set a clean background color (e.g., `#f4f7f6`) and a modern font family like `sans-serif`.
5. Wrap your application form container (`<form>`) inside a `<div>` with a class of `form-card`.
6. Target `.form-card` in your CSS and apply:
    - A white background (`background-color: white;`).
    - A maximum width constraint (`max-width: 500px;`) and center it on the screen (`margin: 40px auto;`).
    - Inner space to keep the input tags breathing (`padding: 30px;`).
    - A clean subtle line wrapper (`border: 1px solid #e0e0e0;`).
7. Save your updates and open your layout in **Live Server** to see your HTML transform visually.
